

Laporan Praktikum Minggu Ke-4 Kontrol Cerdas

Minggu ke-4

Nama : Ulfiana Sofiyanti
NIM : 224308073
Kelas : TKA 7C
Akun Github (Tautan) : <https://github.com/UlfianaSofiyanti>

1. Judul Percobaan

Reinforcement Learning for Autonomus Control

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

- a. Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- b. Mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
- c. Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- d. Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- e. Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori

Reinforcement Learning (RL) didefinisikan sebagai metodologi pembelajaran mesin di mana agen (entitas pembuat keputusan) secara otomatis mengoptimalkan strateginya melalui interaksi berkelanjutan dengan lingkungan (*environment*). Berbeda dari *supervised learning* yang bergantung pada data berlabel yang sudah ada, agen RL belajar secara mandiri dengan menerima umpan balik berupa imbalan (*reward*) atau hukuman (*penalty*) dari lingkungan setelah melakukan serangkaian aksi. Proses pembelajaran ini bersifat iteratif, didorong oleh mekanisme *trial-and-error* dan akumulasi pengalaman, dengan tujuan utama untuk menemukan kebijakan yang memaksimalkan total imbalan positif yang diperoleh seiring waktu (Andreanus and Kurniawan, 2018).

Unsur-unsur Penting dalam Sistem RL

Sistem RL tersusun atas beberapa unsur fundamental yang saling berinteraksi, membentuk siklus pembelajaran:

- a) **Agent:** Merupakan entitas yang memiliki kemampuan untuk melakukan serangkaian aksi. Agen dapat diimplementasikan sebagai perangkat lunak (*software*) atau perangkat keras (*hardware*), bergantung pada sifat dan kompleksitas lingkungan yang dipelajari.
- b) **Environment:** Adalah lingkungan yang dijadikan sebagai objek pembelajaran bagi agen. *Environment* menerima aksi dari agen dan memprosesnya untuk menghasilkan *state* baru serta *reward*.
- c) **State:** Mewakili kondisi atau keadaan spesifik *environment* pada saat diamati oleh agen. Nilai *state* ini akan berubah seiring waktu sebagai respons terhadap aksi yang dipilih oleh agen.
- d) **Action:** Merupakan sekumpulan kemungkinan pergerakan atau langkah diskrit maupun kontinu yang dapat dilakukan oleh agen dalam menjelajahi *environment*.
- e) **Reward:** Adalah sinyal penghargaan atau hukuman yang diterima agen sebagai umpan balik dari *environment* setelah melakukan aksi. *Reward* ini krusial karena menjadi tolok ukur yang memandu agen untuk mengoptimalkan strateginya.

Deep Q-Network (DQN) merupakan inovasi signifikan dari algoritma Q-Learning klasik yang termasuk dalam paradigma model-free. Dalam konteks ini, *model-free* mengasumsikan bahwa agen beroperasi tanpa pengetahuan eksplisit mengenai model matematis lingkungan, seperti probabilitas transisi antar *state* atau nilai *reward* yang akan diterima. Oleh karena itu, agen DQN membangun "pengetahuan" dari nol, murni melalui pengalaman interaksi.

Fungsi inti dari algoritma ini adalah penggunaan jaringan saraf tiruan (*deep neural network*) yang berfungsi sebagai *function approximator*. Jaringan ini menggantikan peran tabel nilai Q (*Q-Table*)—yang menjadi tidak efisien untuk lingkungan berdimensi tinggi—untuk secara efisien mengaproksimasi nilai-Q (*Q-value*), yaitu nilai harapan imbalan yang akan didapatkan jika agen mengambil aksi tertentu pada *state* tertentu.

4. Analisis dan Diskusi

Analisis

- Bagaimana performa agen dalam mengontrol environment CartPole?

Kinerja agen **Deep Q-Network (DQN)** menunjukkan peningkatan signifikan seiring berjalannya sesi pelatihan. Awalnya, agen sering gagal mencapai stabilitas dalam waktu singkat. Namun, melalui proses eksplorasi (*trial-and-error*) selama banyak *episode*, agen berhasil mempelajari strategi kontrol yang efektif. Hal ini ditunjukkan dengan kemampuan agen untuk menjaga keseimbangan tiang dalam durasi yang jauh lebih lama, bahkan mendekati batas maksimum yang ditetapkan (500 langkah). Hal ini membuktikan bahwa agen berhasil belajar strategi kontrol yang efektif melalui proses *trial-and-error*.

- Bagaimana perubahan parameter (misal: gamma, epsilon, learning rate) mempengaruhi kinerja agen?

Kinerja agen sangat dipengaruhi oleh penyesuaian hyperparameter kunci. Nilai Gamma (γ) yang lebih tinggi mengarahkan agen untuk memprioritaskan reward jangka panjang (*long-term rewards*), menghasilkan strategi yang lebih stabil meskipun membutuhkan waktu pelatihan yang lebih lambat. Sementara itu, Epsilon (ϵ) mengatur rasio eksplorasi (*exploration*). Nilai *epsilon* yang tinggi di awal pelatihan mendorong agen untuk menjelajahi berbagai aksi, yang kemudian diturunkan secara bertahap untuk beralih ke fase eksploitasi (*exploitation*) strategi optimal. Terakhir, Learning Rate (α) menentukan kecepatan pembaruan bobot (*weight updates*); nilai yang terlalu tinggi dapat menyebabkan ketidakstabilan dalam pembelajaran, sedangkan nilai yang terlalu rendah akan memperlambat konvergensi menuju solusi yang optimal.

- Apa tantangan yang muncul selama pelatihan agen RL?

Tantangan krusial dalam pelatihan agen *Reinforcement Learning* (RL) adalah kestabilan pelatihan. Ketidakadaan jaringan target (*target network*) menyebabkan nilai Q tidak stabil karena pembaruan prediksi dan target dilakukan oleh model yang identik. Selain itu, proses *training*

menuntut waktu komputasi yang signifikan karena membutuhkan banyak *episode*. Agen juga berisiko tinggi terjebak pada solusi lokal jika eksplorasi tidak dilaksanakan secara memadai.

Diskusi

- Apa perbedaan utama antara Reinforcement Learning dan metode supervised learning dalam sistem kendali?

Perbedaan yang mendasar terletak pada **sumber pengetahuan**. *Supervised learning* memerlukan **data berlabel** yang telah disiapkan sebelumnya, sementara *Reinforcement Learning* (RL) belajar secara **langsung dari interaksi** lingkungan melalui *reward* dan *penalty* tanpa memerlukan label data. Oleh karena sifatnya yang dinamis, RL lebih efektif untuk sistem kendali otonom yang **sulit diprediksi**.

- Bagaimana strategi eksplorasi (exploration) dan eksploitasi (exploitation) dapat dioptimalkan pada agen RL?

Untuk mengoptimalkan **eksplorasi (exploration)** dan **eksploitasi (exploitation)**, digunakan pendekatan *epsilon-greedy*. Strategi ini dimulai dengan **nilai ϵ yang tinggi** untuk mendorong eksplorasi yang luas, kemudian **ϵ diturunkan secara bertahap** agar agen fokus mengeksploitasi strategi terbaik yang telah ditemukan. Metode alternatif yang dapat digunakan untuk menyeimbangkan kedua aspek ini termasuk **Boltzmann Exploration** atau **Upper Confidence Bound (UCB)**.

- Potensi aplikasi lain dari RL dalam sistem kendali nyata apa saja yang dapat diimplementasikan?

Reinforcement Learning (RL) menawarkan **potensi aplikasi yang luas** dalam sistem kendali nyata karena kemampuannya beradaptasi dengan kondisi dinamis. Implementasi RL meliputi: **navigasi robot** di lingkungan yang kompleks, **pengendalian kendaraan otonom**, **pengaturan energi** pada *smart grid*, **optimasi logistik dan manajemen gudang**, serta **otomasi proses industri**. Pemanfaatan RL memungkinkan terciptanya sistem kendali yang jauh **lebih cerdas dan efisien**.

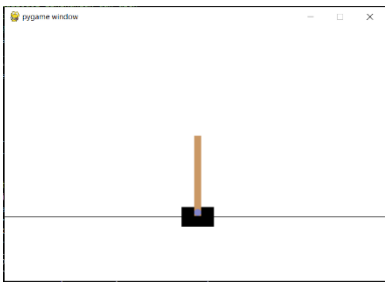
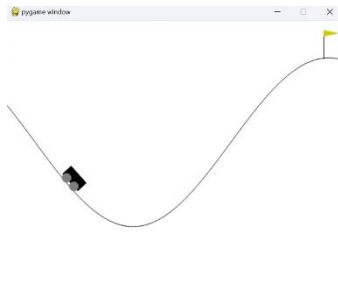
5. Assignment

Algoritma Deep Q-Network (DQN) dimodifikasi dengan mengintegrasikan jaringan target (*target network*) guna meningkatkan stabilitas selama proses pelatihan. Implementasi *target network* ini memastikan bahwa pembaruan nilai-Q berlangsung secara lebih konsisten, memungkinkan agen untuk belajar secara optimal tanpa mengalami osilasi yang signifikan.

Kemudian Agen dievaluasi secara komprehensif pada tiga lingkungan simulasi yang berbeda: *CartPole-v1*, *LunarLander-v2*, dan *MountainCar-v0*. Hasil pengujian menunjukkan adanya variasi tingkat kesulitan di antara lingkungan tersebut. Agen dengan cepat menguasai tugas menjaga keseimbangan pada *CartPole*. Sebaliknya, *LunarLander* memerlukan jumlah *episode* pelatihan yang lebih tinggi agar agen dapat mengembangkan strategi pendaratan yang efektif. Sementara itu, *MountainCar* menantang agen untuk memanfaatkan momentum sebagai kunci untuk mencapai puncak bukit. sehingga proses belajar akan lebih lambat.

6. Data dan Output Hasil Pengamatan

No	Parameter	CartPole-v1	MountainCar-v0
1.	Waktu Pelatihan	Agen mampu mencapai performa optimal dengan efisiensi tinggi, hanya memerlukan beberapa ratus <i>episode</i> pelatihan untuk menyelesaikan tugas	Agen membutuhkan lebih banyak <i>episode</i> karena lingkungan lebih kompleks dan <i>reward</i> jarang didapat.
2.	Jumlah Episode	Agen menunjukkan kemampuan menjaga keseimbangan secara stabil pada batas 1000 <i>episode</i> . <pre>if __name__ == "__main__": env = gym.make("CartPole-v1", render_mode=None) state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32</pre>	MountainCar di training hingga 1000 episode <pre>if __name__ == "__main__": # pakai _name_ env = gym.make("MountainCar-v0", render_mode="human") state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32</pre>

3.	Skor per Episode (Testing)	Skor rata-rata meningkat signifikan (contoh : 8, 9, 10, 11 hingga >200) <pre> Episode 2: Score = 9 Episode 3: Score = 10 Episode 4: Score = 9 Episode 5: Score = 10 Episode 6: Score = 10 Episode 7: Score = 10 Episode 8: Score = 8 Episode 9: Score = 9 Episode 10: Score = 11 </pre>	Skor awal mencapai 199 karena maksimal langkah dalam satu episode adalah 200 <pre> Episode: 1/1000, Score: 199, Epsilon: 1.00 Episode: 2/1000, Score: 199, Epsilon: 0.99 Episode: 3/1000, Score: 199, Epsilon: 0.99 Episode: 4/1000, Score: 199, Epsilon: 0.99 </pre> <pre> Episode: 14/1000, Score: 199, Epsilon: 0.94 Episode: 15/1000, Score: 199, Epsilon: 0.93 Episode: 16/1000, Score: 199, Epsilon: 0.93 Episode: 17/1000, Score: 199, Epsilon: 0.92 Episode: 18/1000, Score: 199, Epsilon: 0.92 Episode: 19/1000, Score: 199, Epsilon: 0.91 Episode: 20/1000, Score: 199, Epsilon: 0.91 </pre>
4.	Kinerja Agen	Agen Semakin Stabil, Skor semakin tinggi	Agen masin menghadapi tantangan untuk mencapai puncak bukit
5.	Kondisi Berhasil	Episode berakhir jika tiang jatuh atauu batas Langkah maksimum tercapai	Episode berakhir jika mobil berhasil sampai puncak bukit
6.	Output		

7. Kesimpulan

Secara keseluruhan, algoritma Deep Q-Network (DQN) terbukti efektif dalam mengendalikan lingkungan simulasi dengan karakteristik yang bervariasi. CartPole-v1 memungkinkan agen mencapai kinerja optimal dengan cepat berkat pola *reward* yang konsisten. Sebaliknya, MountainCar-v0 menuntut jumlah *episode* pelatihan yang lebih besar dan eksplorasi yang lebih mendalam. Temuan ini menegaskan bahwa kompleksitas lingkungan secara langsung meningkatkan tantangan yang harus diatasi dalam pembelajaran *Reinforcement Learning* (RL).

8. Saran

Menerapkan jaringan target (*target network*) dan meningkatkan kapasitas memori *experience replay*. Langkah ini akan memastikan konsistensi pembaruan nilai Q dan meningkatkan stabilitas pelatihan. Selain itu, penting untuk memperluas kajian dengan membandingkan algoritma *Reinforcement Learning* (RL) alternatif seperti Policy Gradient, Actor-Critic, atau PPO, guna memahami keunggulan relatifnya di lingkungan yang lebih kompleks.

9. Daftar Pustaka

Andreanus, J., Kurniawan, A., 2018. Sejarah, Teori Dasar dan Penerapan Reinforcement Learning: Sebuah Tinjauan Pustaka. J. Telematika 12, 113–118. <https://doi.org/10.61769/telematika.v12i2.193>